

PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2

[Página inicial](#)

[Meus cursos](#)

[PROJETO E ANÁLISE DE ALGORITMOS – 01A – 2022.2](#)

[Tópico 15](#)

[Notas de aula: NP-completude](#)

Notas de aula: NP-completude

Motivação

A maior parte dos problemas que estudamos podem ser resolvidos em **tempo polinomial** ($O(n^k)$).

- Não foi o caso dos pseudo-polinomiais.

Os problemas que possuem tempo polinomial são chamados de **tratáveis**, e os que possuem tempo com classe acima da polinomial são chamados de **intratáveis** (ou difíceis).

- Podemos ter um polinômio de grau muito alto, que também tornaria o problema difícil de resolver. Porém, este caso é raro.
- Problemas cuja saída é exponencial no tamanho da entrada são intratáveis. Por exemplo, algoritmo para listar todos os subconjuntos, ou todas as permutações.

Existe uma classe grande de problemas (**NP-completos**) onde nenhum algoritmo polinomial foi encontrado, e ninguém foi capaz de provar que tal algoritmo não existe.

- Encontrar algoritmo polinomial para um problema NP-completo significa encontrar algoritmo polinomial para todos os problemas NP-completos.
- Os cientistas da computação acreditam que esta classe é intratável, pois muitos pesquisadores já tentaram encontrar algoritmo polinomial para problemas NP-completo.
- Isto é um problema em aberto desde 1971.
- Portanto, se provarmos que o problema que estamos trabalhando é NP-completo, podemos afirmar que a chance deste problema ser tratável é muito pequena.



Uma aspecto frustrante de problemas NP-completos é que muitos deles são na verdade pequenas variações sobre problemas tratáveis.

Por exemplo,

- Encontrar um caminho mais curto em um grafo é tratável, porém encontrar um **caminho mais longo** é NP-completo. É NP-completo mesmo quando todas as arestas têm custo 1.
- Encontrar ciclo que passa exatamente 1x em cada aresta é tratável (trajeto de Euler). Porém, é NP-completo se deve passar exatamente 1x em cada vértice (**ciclo hamiltoniano**).
- Uma fórmula booleana é k -CNF se for o AND de cláusulas de OR de exatamente k variáveis ou suas negações. Por exemplo, $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_3)$ é 2-CNF. O problema de **satisfatibilidade** consiste em determinar se existe atribuição de valor lógico para as variáveis de modo a tornar a fórmula verdadeira. 2-CNF é tratável, 3-CNF é NP-completo.
- Encontrar uma árvore geradora de custo mínimo (AGM) é tratável, porém encontrar uma AGM onde o grau de cada nó não folha é no mínimo/máximo k é NP-completo.

Um bom projetista de algoritmos deve conhecer o básico da teoria dos problema NP-completos.

- Mostrando que um problema é NP-completo, você fornece forte evidência de que o problema é intratável.
- Portanto, é melhor gastar tempo buscando uma solução aproximada do que buscar um algoritmo rápido que encontre a solução ótima.
- Além disso, geralmente é mais fácil provar que um problema é NP-completo do que estabelecer um limite inferior para todo algoritmo para o problema. Por exemplo, a demonstração de que todo algoritmo de ordenação é $\Omega(n \log n)$.

Problemas de decisão *versus* problemas de otimização

Seja S o conjunto de soluções possíveis para um problema, e seja $f : S \rightarrow \mathbb{R}$ uma função que fornece a qualidade de cada solução.

Em um **problema de otimização**, desejamos encontrar uma solução s^* tal que $f(s^*) \leq f(s)$ para todo $s \in S$ (problema de minimização).

Todo problema em que a resposta é simplesmente "sim" ou "não" é chamado **problema de decisão**.

Para todo problema de otimização temos um problema de decisão associado: podemos perguntar se existe solução s tal que $f(s) \leq k$, para alguma constante k .

- Por exemplo, encontrar caminho mínimo é um problema de otimização. O problema que responde "sim" apenas se existe caminho de custo no máximo 10 é um problema de decisão.

Quando o problema de otimização é tratável, sua versão de decisão também será: podemos executar o algoritmo de otimização para responder.

Portanto, quando um problema de decisão é intratável, sua versão de otimização também será (contra-positiva).

De agora em diante consideramos apenas problemas de decisão.

Problemas de decisão são facilmente modelados com linguagens formais: conjuntos de strings de entrada que produzem "sim" é uma linguagem.

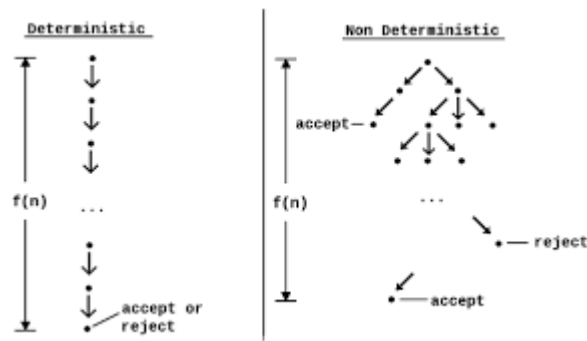
Classes de problemas P, NP e NPC

Um problema está na **classe P** se pode ser resolvido em tempo polinomial. Ou seja, existe algoritmo de tempo $O(n^k)$ para alguma constante k , onde n é o tamanho da entrada.

A **classe NP** é formada por problemas que podem ser "verificados" em tempo polinomial. Ou seja, dada uma solução s (certificado), podemos testar em tempo polinomial se $f(s) \leq k$.

- Por exemplo, dado um caminho no grafo, podemos calcular seu custo em tempo polinomial (somando o peso de suas arestas). Podemos também verificar em tempo polinomial que forma um caminho. Neste caso, o problema de caminho mais longo também pertence a NP.
- Dado um ciclo no grafo (sequência de vértices $v_1, v_2, \dots, v_k$), podemos verificar em tempo polinomial se forma um ciclo hamiltoniano: deve existe aresta de v_i para v_{i+1} , para todo i . Também deve existir aresta de v_k para v_1 . Cada vértice do grafo deve aparecer neste sequência exatamente uma vez. Toda esta verificação pode ser feita em tempo polinomial. Logo, o problema de decidir se uma solução é um ciclo hamiltoniano pertence a NP.
- Note que NP não significa "não polinomial", como muitos acreditam. Na verdade, NP significa que o problema pode ser resolvido em tempo polinomial por uma máquina não determinística (de onde vem o N).

- Uma máquina não determinística pode explorar a árvore de possibilidades em paralelo, e uma delas é o certificado. Não existe na prática, mas dispomos de muitos resultados teóricos.



Todo problema em P pertence a NP.

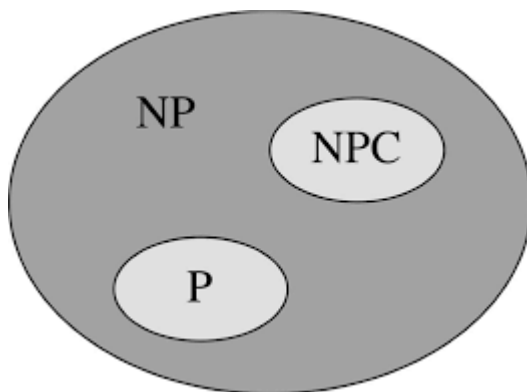
Quando o problema pertence a P, sabemos que ele responde em tempo polinomial considerando todos os possíveis certificados (soluções possíveis).

A pergunta em aberto desde 1971 é se todo problema em NP pertence a P (ou seja, $P = NP$). Não sabemos se resolver um problema é tão difícil quanto testar um certificado!

A **classe NPC** (NP-completo) é o conjunto de problemas mais difíceis dentro da classe NP.

Encontrar algoritmo polinomial para qualquer problema em NPC significa encontrar algoritmo polinomial para toda a classe NP.

Isto responderia a pergunta $P=NP$.



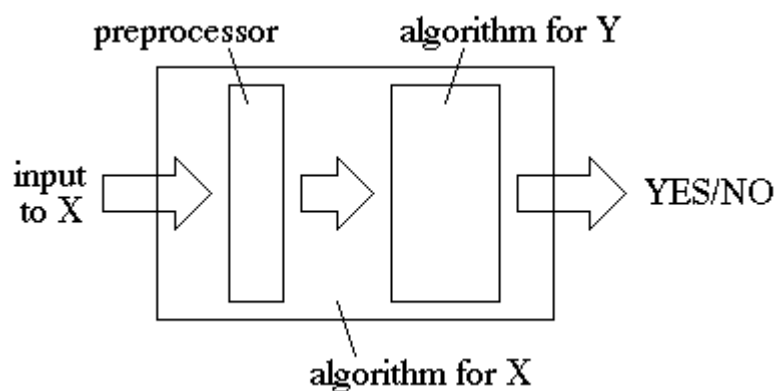
Se existir problema Y em NP e fora de P e NPC , então $P \neq NP$.

Problema polinomial X em NPC poderia ser usado para resolver Y em tempo polinomial.

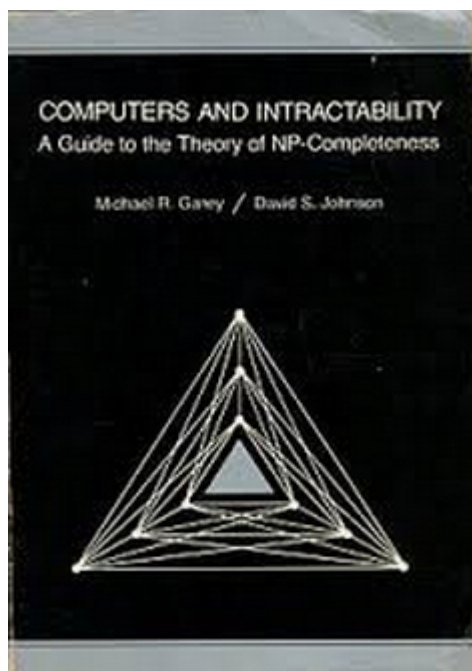
Como mostrar que um problema é NP-completo?

Passos para mostrar que o problema Y é NP-completo:

1. Mostre que Y pertence a NP.
2. Identifique um problema NP-completo X que facilite os passos seguintes. Consulte [Garey & Jonhson].
3. Forneça algoritmo A que transforma qualquer entrada de X em uma entrada de Y , tal que Y responde "sim" sse X também responde "sim". O algoritmo A deve ser polinomial no tamanho da entrada de X .



Catálogo de problemas NP-completos:



Se conseguirmos seguir todos os passos, concluímos que

- Algoritmo polinomial para Y resolveria toda a classe NP em tempo polinomial, pois podemos usar a transformação para resolver o problema NP-completo X em tempo polinomial.
- Ou seja, Y também é NP-completo.
- Logo, provavelmente não existe algoritmo polinomial para Y .

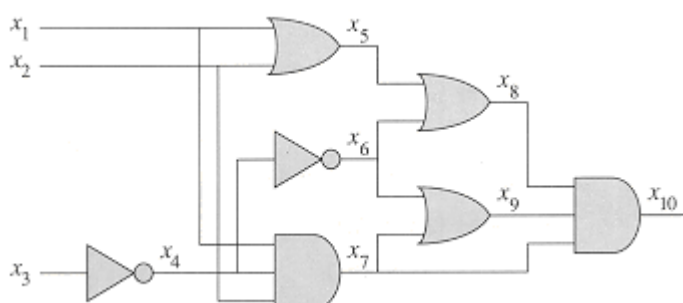
Um erro comum é realizar transformação de instâncias (algoritmo A) em tempo exponencial.

- Imagine que um inteiro k faz parte da entrada de X . Se, por exemplo, criarmos k nós em um grafo de entrada para Y , estamos fazendo uma transformação exponencial.
- Isto porque o número de nós criados cresce exponencialmente no número de bits que representam o valor de k .

O primeiro problema NP-completo

Os passos mostrados anteriormente assumem que já temos pelo menos um problema NP-completo.

O primeiro problema provado como NP-completo foi a satisfatibilidade de circuitos: dado um circuito booleano com portas AND, OR e NOT, existe um conjunto de entradas booleanas de modo que a saída seja 1?



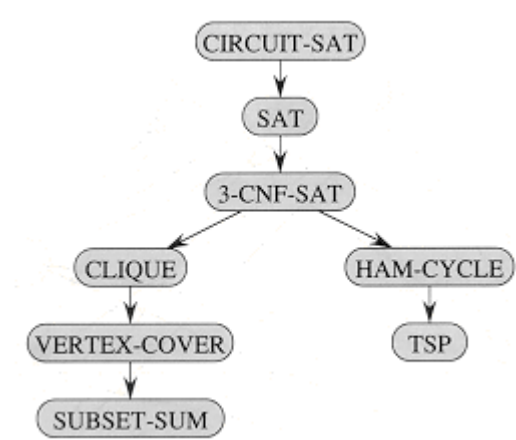
Como computadores são construídos com circuitos lógicos, é natural imaginar que é possível simular a execução de máquinas determinísticas para resolver problemas de decisão usando satisfatibilidade de circuitos.

- Ou seja, para cada problema de decisão em P , criamos em tempo polinomial um circuito que seja satisfatível sse o problema responde "sim".
- Deve ser possível, pois o computador executa operações lógicas sequenciais para resolver o problema de decisão em tempo polinomial. De fato é possível provar que sim.

A surpresa foi a prova [Cook-Levin,1971] de que a satisfatibilidade de circuitos simula máquinas **não determinísticas** de tempo polinomial (problemas da classe NP).

- Ou seja, todo algoritmo para problema de decisão que executa em tempo polinomial em máquina não determinística pode ser transformado em tempo polinomial no problema de satisfatibilidade de circuitos.
- Prova não é complexa, mas muito trabalhosa.

Exemplos de provas de NP-completude



3-CNF-SAT

Pertence a NP, pois dada atribuição de valores lógicos às variáveis, podemos verificar em tempo polinomial que satisfaz a fórmula.

Resta colocar todas as cláusulas com 3 variáveis.

Se tem menos de 3, repetimos uma das variáveis:

$(\neg x_1 \vee x_2)$ torna-se $(\neg x_1 \vee x_2 \vee x_2)$.

Podemos transformar cláusula com 4 variáveis em 2 com 3:
 $(x_1 \vee x_2 \vee x_3 \vee x_4)$ é o mesmo que $(x_1 \vee x_2 \vee y) \wedge (\neg y \vee x_3 \vee x_4)$.

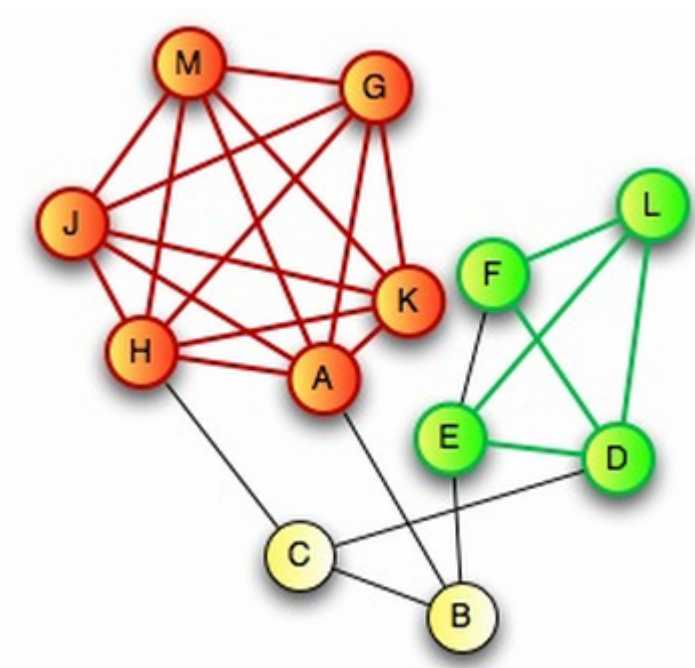
De forma geral podemos substituir $(x_1 \vee x_2 \vee \dots \vee x_n)$ por
 $(x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge (\neg y_2 \vee x_4 \vee y_3) \wedge \dots \wedge (\neg y_{n-3} \vee x_{n-1} \vee x_n)$.

Para cada cláusula de tamanho n , produzimos $n - 3$ novas cláusulas de tamanho 3, e criamos $n - 3$ novas variáveis. Portanto, a transformação é polinomial.

Clique em grafos

Uma **clique** em grafo não orientado $G(V, E)$ é um subconjunto V' de V tal que, para cada par u, v em V' , temos que $(u, v) \in E$.

- Subgrafo induzido por V' forma um grafo completo.



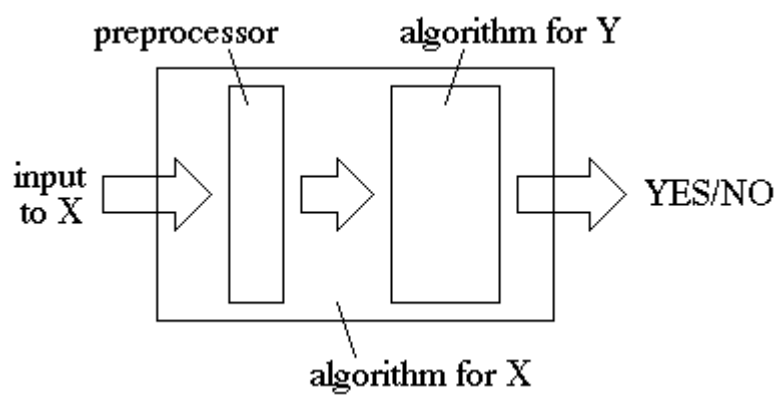
Problema CLIQUE: existe uma clique com pelo menos k vértices?

CLIQUE pertence a NP: podemos verificar em tempo polinomial que um certificado (conjunto de vértices V') forma uma clique de tamanho pelo menos k .

- Verificamos se V' tem pelo menos k elementos.
- Para cada par de vértices $u, v \in V'$, verificamos se $(u, v) \in E$.

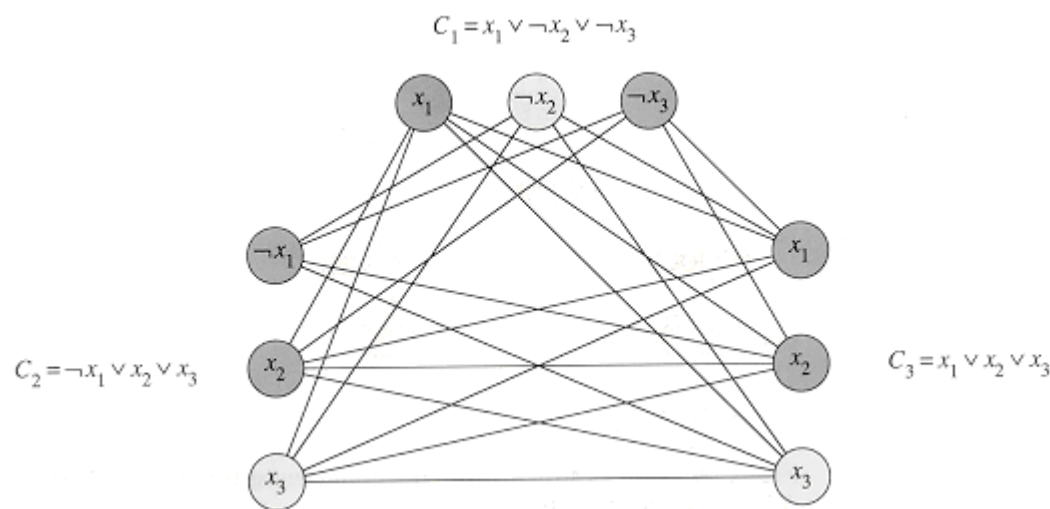
O problema NP-completo escolhido para a prova é o 3-CNF, o que é uma surpresa, pois não parece ter relação com grafos.

- Transformamos uma expressão com k cláusulas em um grafo, de modo que uma clique neste grafo fornece uma atribuição de valores às variáveis que satisfazem a expressão booleana.



Algoritmo para transformação de instâncias:

1. Para cada cláusula, crie grupo de 3 vértices, cada um representando um elemento da cláusula (var. ou sua negação).
2. Crie arestas entre todos os pares de vértices de cláusulas distintas que são logicamente compatíveis.



Como temos k grupos de 3 vértices, e as arestas existem apenas entre vértices compatíveis em grupos diferentes, se existir uma clique de tamanho k neste grafo, podemos concluir que existe uma atribuição de valores válida.

- Podemos tornar todos os vértices na clique com valor verdadeiro, satisfazendo todas as cláusulas da expressão booleana.

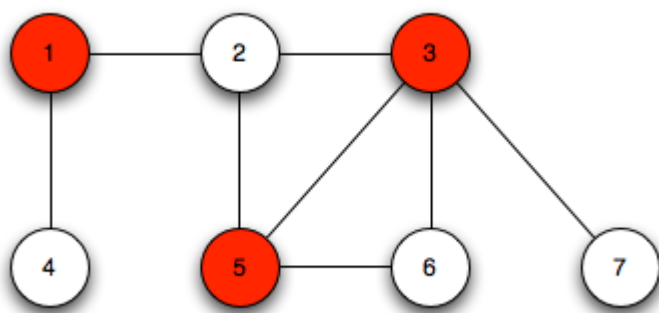
Na verdade estamos mostrando que decidir se existe clique de tamanho k em grafos organizado em k grupos de 3 vértices, onde não existe aresta dentro de um mesmo grupo, é NP-completo.

- Entretanto, como um algoritmo que decide se existe clique de tamanho k em um grafo qualquer é capaz de decidir sobre este grafo particular, concluímos que o problema CLIQUE é NP-completo.

Cobertura por vértices

Uma **cobertura por vértices** de um grafo não orientado $G(V, E)$ é um conjunto de vértice V' tal que, para toda aresta, pelo menos um dos vértice u e v pertence a V' .

Ou seja, cada aresta incide em pelo menos um vértice em V' .



Problema COBERTURA: é possível cobrir as arestas de G com no máximo k vértices?

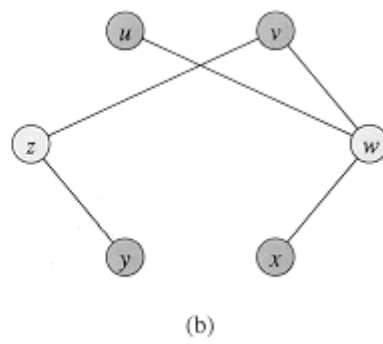
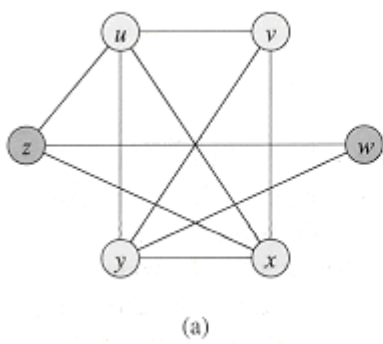
COBERTURA pertence a NP, pois em tempo polinomial determinamos que

1. V' tem no máximo k elementos.
2. Para cada aresta $(u, v) \in E$, temos $u \in V'$ ou $v \in V'$.

Vamos utilizar o problema CLIQUE para mostrar que COBERTURA é NP-completo: vamos decidir se o grafo tem clique de tamanho k verificando se um determinado grafo criado em tempo polinomial tem cobertura de tamanho k' .

Algoritmo para transformação da entrada: obtenha o grafo complemento de G .

- O **grafo complemento** $\overline{G}(V, \overline{E})$ é o grafo formado por todas os vértices de V , e todas as arestas que não estão em E .



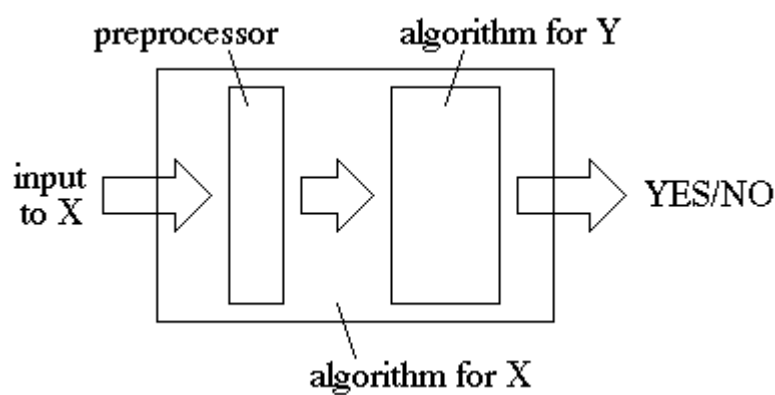
C é uma clique em G sse $V - C$ é uma cobertura em $\overline{G}$.

- Se C é clique em G , então $V - C$ é cobertura em $\overline{G}$, pois não existe aresta entre vértice de C em $\overline{G}$.
- Se V' é cobertura em $\overline{G}$, então $V - V'$ é uma clique de G , pois caso contrário haveria aresta descoberta.

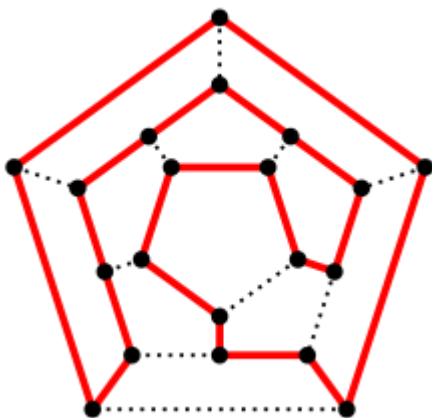
Portanto, o grafo G tem clique de tamanho pelo menos k sse $\overline{G}$ tem cobertura de tamanho no máximo $|V| - k$.

Se for possível decidir em tempo polinomial se $\overline{G}$ tem cobertura de tamanho no máximo $|V| - k$, então é possível decidir em tempo polinomial se G tem clique de tamanho pelo menos k .

Assim, COBERTURA é NP-completo.



Caixeiro viajante



Vamos utilizar o problema HAMILTONIANO para mostrar que o problema CAIXEIRO_VIAJANTE é NP-Completo. Assuma que o problema HAMILTONIANO é NP-Completo.

HAMILTONIANO: Dado um grafo não-direcionado G , ele contém ciclo hamiltoniano?

Um ciclo hamiltoniano é um ciclo que passa exatamente uma vez em cada vértice, ou seja, todos os vértices no grafo incidem em exatamente duas arestas do ciclo.

CAIXEIRO_VIAJANTE: Dado um grafo não-direcionado completo G com pesos não-negativos nas arestas, e um limite k , existe um ciclo que passa em todos os vértices e possui custo no máximo k ?

No problema do **CAIXEIRO_VIAJANTE** o grafo tem pesos nas arestas, e desejamos determinar se existe ciclo hamiltoniano com custo menor ou igual a k .

O problema CAIXEIRO_VIAJANTE pertence a NP: em tempo polinomial determinamos se as arestas formam um ciclo hamiltoniano e somamos seus pesos (comparando assim com k). Verificar que as arestas no certificado formam um ciclo hamiltoniano pode ser feito em tempo polinomial: (i) podemos utilizar o fato do problema HAMILTONIANO ser NP-Completo (logo pertence a NP, e seu certificado pode ser verificado em tempo polinomial), ou (ii) podemos fazer um algoritmo polinomial que verifica que exatamente duas arestas incidem em cada vértice e que as arestas formam uma única componente conexa (basta fazer uma busca no grafo).

Vamos usar o problema HAMILTONIANO para mostrar que CAIXEIRO_VIAJANTE é NP-completo.

Transformação da entrada: criamos um grafo completo G' contendo os mesmos vértices de V .

O custo de cada arestas (u, v) é dado por

$$c(u, v) = \begin{cases} 0, & \text{se } (u, v) \in E \\ 1, & \text{caso contrário} \end{cases}$$

Desta forma, o algoritmo para CAIXEIRO_VIAJANTE retorna uma solução de custo zero sse G tem um ciclo hamiltoniano.

◀ [Slides: NP-completude](#)

Seguir para...

[TRABALHO: definição das equipes](#) ▶

©2020 - Universidade Federal do Ceará - Campus Quixadá.

Todos os direitos reservados.

Av. José de Freitas Queiroz, 5003

Cedro - Quixadá - Ceará CEP: 63902-580

Secretaria do Campus: (88) 3411-9422

📱 [Baixar o aplicativo móvel.](#)